

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-adc569f0875b888b9.jsonl`
- SHA-256: `85649327efa1149944b4a5c3ca4c8571c4b7eebc9317c38fff9c32181552fe30`
- Source modified: `2026-06-22T09:47:08+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:45:35.965Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "Cost report uses rate_per_sec without validating machine_usd_per_hr is positive",
  "file": "scripts/nightly_reelcount.py",
  "line": "79",
  "severity": "low",
  "detail": "At line 79, rate_per_sec = float(machine_usd_per_hr) / 3600.0. If machine_usd_per_hr is negative or zero, the cost will be zero or negative. The manifest provides a default (line 59: DEFAULT_MACHINE_USD_PER_HR = 0.35), but manifest.get('machine_usd_per_hr', ...) on line 497 could receive a malformed value. No validation is performed. If machine_usd_per_hr is 0, the cost is $0 (misleading but not a crash). If it's negative (typo in manifest), the cost is negative (clearly wrong).",
  "fix": "Add validation in cost_report() or main(): `if machine_usd_per_hr <= 0: raise ValueError(f'machine_usd_per_hr must be > 0, got {machine_usd_per_hr}')` immediately after line 497."
}
```

2. assistant (2026-06-22T09:45:37.970Z)

I'll analyze the nightly regression change by examining the relevant files to verify the finding about the cost report validation issue.

3. user (2026-06-22T09:45:39.432Z)

```
1      """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3      Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4      it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
```

```

5      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7      needs a GPU ? hence it runs on an ephemeral GCP GPU VM
8      (`deploy/gcp/nightly_regression/``), emails the
9      owner the report, and self-deletes. It is default-additive: a tripwire, not the
promotion gate.
10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     EXACT (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")

```

```

53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02      # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                          cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
109             }

```

```

110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
111                 "n": len(per_video)}
112
113
114     @dataclass
115     class Finding:
116         video: str
117         metric: str                # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
118         kind: str                # "regression" | "improvement" | "stale"
119         baseline: Optional[float] = None
120         current: Optional[float] = None
121         detail: str = ""
122
123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134     @dataclass
135     class NightlyResult:
136         findings: List[Finding] = field(default_factory=list)
137         added: List[str] = field(default_factory=list)
138         removed: List[str] = field(default_factory=list)
139
140     @property
141     def regressions(self) -> List[Finding]:
142         return [f for f in self.findings if f.kind == "regression"]
143
144     @property
145     def improvements(self) -> List[Finding]:
146         return [f for f in self.findings if f.kind == "improvement"]
147
148     @property
149     def stale(self) -> List[Finding]:
150         return [f for f in self.findings if f.kind == "stale"]
151
152     def overall_status(self) -> str:
153         if self.regressions:
154             return "REGRESSION"
155         if self.stale:
156             return "STALE"
157         return "PASS"
158
159     def exit_code(self) -> int:
160         if self.regressions:
161             return 1
162         if self.stale:
163             return 4
164         return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.

```

```

172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
179         return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))
184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",
186                                     detail=f"added={res.added} removed={res.removed}"))
187
188         for v in sorted(set(b_pv) & set(c_pv)):
189             b, c = b_pv[v], c_pv[v]
190             # 1) pipeline must still succeed
191             if not c.get("ok", False) or c.get("reel_count") is None:
192                 res.findings.append(Finding(v, "error", "regression",
193                                     detail=str(c.get("error") or "no reel_count
produced")))
194                 continue
195             # 2) reel count ? EXACT
196             bc, cc = b.get("reel_count"), c.get("reel_count")
197             if bc is not None and cc is not None and cc != bc:
198                 res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199             # 3) quality metrics ? tolerance (only if both sides have them)
200             bq, cq = b.get("quality") or {}, c.get("quality") or {}
201             for m in QUALITY_HIGHER:
202                 bv, cv = bq.get(m), cq.get(m)
203                 if bv is None or cv is None:
204                     continue
205                 if cv < bv - tols.quality_tol:
206                     res.findings.append(Finding(v, m, "regression", bv, cv))
207                 elif cv > bv + tols.quality_tol:
208                     res.findings.append(Finding(v, m, "improvement", bv, cv))
209         return res
210
211
212         # ----- #
213         # Report formatting (pure).
214         # ----- #
215         def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
216             if not qd or qd.get(key) is None:
217                 return "?"
218             return f"{qd[key]:.3f}"
219
220
221         def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                         result: NightlyResult, date: str, head: str) -> str:
223             status = result.overall_status()
224             badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                     "STALE": "? STALE (re-freeze the baseline)"}[status]
226             cost = current.get("cost", {})
227             L: List[str] = [
228                 f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229                 "",
230                 f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
_{current.get('segmenter')}",
231                 "",

```

```

232         f"- HEAD `{head}` · baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
233         f"- **Run cost: ${cost.get('usd', 0):.4f} ({cost.get('inr', 0):.2f})** · "
234         f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
235         "",
236     ]
237     if result.regressions:
238         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
239     if result.stale:
240         L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241         L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", ""]
246         "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247         "|---|---|---|---|---|"
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"_{q}(bq, 'tol_f1')?_{q}(cq, 'tol_f1')}"
255         f5 = f"_{q}(bq, 'f1@0.5')?_{q}(cq, 'f1@0.5')}"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else ""
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
+ [f"- {f.message()}" for f in result.improvements] + [""]
262
263     L += ["_--",
264         "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
265         "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
266     return "\n".join(L)
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:

```

```

287         for row in csv.reader(fh):
288             try:
289                 out.append((float(row[0]), float(row[1])))
290             except (ValueError, IndexError):
291                 continue
292     return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(start, end) pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
298         optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too

```

```

342         db = Database(db_path)
343         video_id = compute_video_id(local)
344         t0 = time.monotonic()
345         result = seg_cls(db, config).process_video(video_id, local)
346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result
350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
367 value
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
372 "tol_precision", "tol_recall")}
373             if k in row}
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
386 total_wall_s)."""
387         from backend.config import load_config
388
389         config = load_config()
390         # apply manifest config overrides onto the typed config (dotted keys)
391         for dotted, val in (manifest.get("config_overrides") or {}).items():
392             _set_dotted(config, dotted, val)
393         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
394 "default_segmenter", "motion")
395         b_pv = (baseline or {}).get("per_video", {})
396         results: List[Dict[str, Any]] = []
397         total_wall = 0.0
398         for v in manifest.get("videos", []):
399             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
400             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
401             total_wall += float(r.get("wall_seconds") or 0.0)
402             results.append(r)
403         return results, total_wall

```

```

403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.
406         'indexing.motion_threshold')."""
407         parts = dotted.split(".")
408         obj = config
409         for p in parts[:-1]:
410             obj = getattr(obj, p, None)
411             if obj is None:
412                 return
413         try:
414             setattr(obj, parts[-1], value)
415         except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
416             crash the run
417             pass
418
419     def _git_head() -> str:
420         import subprocess
421         try:
422             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
423                                 capture_output=True, text=True, timeout=10)
424             return out.stdout.strip() or "?"
425         except Exception: # noqa: BLE001
426             return "?"
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433
434     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
435         out = dict(snapshot)
436         out["generated_at"] = _dt.date.today().isoformat()
437         out["git_commit"] = _git_head()
438         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
439         scripts/nightly_reelcount.py "
440         "--update-baseline` after an INTENDED engine change or when adding a
441         video. Tied to the "
442         "configured+checkpointed engine + the manifest corpus.")
443         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
444         os.makedirs(os.path.dirname(path), exist_ok=True)
445         tmp = path + ".tmp"
446         with open(tmp, "w", encoding="utf-8") as fh:
447             json.dump(out, fh, indent=2, sort_keys=True)
448             fh.write("\n")
449         os.replace(tmp, path)
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
453         quality + cost.")
454         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
455         p.add_argument("--baseline", default=DEFAULT_BASELINE)
456         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
457         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
458         p.add_argument("--vm-uptime-seconds", type=float, default=None,
459             help="authoritative VM uptime (boot?delete) for the cost line; "
460             "defaults to the in-process pipeline wall time (a lower bound)")
461         p.add_argument("--no-rerun-guard", action="store_true",
462             help="disable the re-run-once-on-mismatch guard (the DB-drop flake
463             mitigation)")

```

```

462         p.add_argument("--update-baseline", action="store_true",
463                         help="freeze the current reel counts + quality as the new baseline
instead of checking")
464         p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465         p.add_argument("--email-to", default=None, help="override the report recipient")
466         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468         return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487             except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488                 log.error("run failed: %s", e)
489                 return 2
490             if not run_results:
491                 log.error("no videos in manifest %s", args.manifest)
492                 return 2
493
494             billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495             cost = cost_report(
496                 billed_seconds=billed,
497                 machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498                 fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499                 gpu_seconds=total_wall,
500             )
501             segmenter = manifest.get("segmenter", "motion")
502             snapshot = summarize_results(run_results, segmenter, cost)
503             snapshot["git_commit"] = _git_head()
504             snapshot["generated_at"] = date
505
506             if args.update_baseline:
507                 save_baseline(args.baseline, snapshot)
508                 log.info("baseline frozen ? %s", args.baseline)
509                 for r in run_results:
510                     log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511                 return 0
512
513             if baseline is None:
514                 log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515                 return 3
516

```

```

517     result = compare(baseline, snapshot, tols)
518     report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520     if not args.no_report:
521         os.makedirs(args.report_dir, exist_ok=True)
522         md_path = os.path.join(args.report_dir, f"{date}.md")
523         with open(md_path, "w", encoding="utf-8") as fh:
524             fh.write(report + "\n")
525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                        "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                        fh, indent=2, sort_keys=True)
529         log.info("report ? %s", md_path)
530
531     print(report)
532
533     if args.email:
534         try:
535             from scripts.nightly_email import send_report
536             subject = f"[KheISutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537             sent = send_report(subject, report, to=args.email_to)
538             log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539         except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540             log.error("email failed (report still written): %s", e)
541
542     code = result.exit_code()
543     if code == 1:
544         log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545     elif code == 4:
546         log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547     else:
548         log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549     return code
550
551
552 if __name__ == "__main__":
553     raise SystemExit(main())
554

```

4. user (2026-06-22T09:45:39.545Z)

```

1     {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6             "indexing.default_segmenter": "wasb_hybrid",
7             "indexing.detector_impl": "native"
8         },

```

```

9      "machine_usd_per_hr": 0.70,
10     "fx_inr_per_usd": 83.0,
11     "videos": [
12         {
13             "name": "GX030094",
14             "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX030094.mp4",
15             "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16         },
17         {
18             "name": "Badminton_BXH_2",
19             "uri": "gs://khehsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20             "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21         },
22         {
23             "name": "GX010128",
24             "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX010128.mp4",
25             "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26         }
27     ]
28 }
29

```

5. assistant (2026-06-22T09:45:41.969Z)

Now let me read the specific lines mentioned in the finding (around line 79 and line 497):

6. user (2026-06-22T09:45:43.580Z)

```

73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                     fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds x machine rate ? USD + INR (`backend.billing`
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92

```

7. user (2026-06-22T09:45:44.689Z)

```

490     if not run_results:
491         log.error("no videos in manifest %s", args.manifest)
492         return 2
493
494     billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495     cost = cost_report(

```

```

496         billed_seconds=billed,
497         machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498         fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499         gpu_seconds=total_wall,
500     )
501     segmenter = manifest.get("segmenter", "motion")
502     snapshot = summarize_results(run_results, segmenter, cost)
503     snapshot["git_commit"] = _git_head()
504     snapshot["generated_at"] = date

```

8. assistant (2026-06-22T09:45:46.766Z)

Now let me check the constants and manifest validation:

9. user (2026-06-22T09:45:47.750Z)

```

50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02      # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL

```

10. user (2026-06-22T09:45:48.785Z)

```

475     args = build_parser().parse_args(argv)
476     tols = Tolerances(quality_tol=args.quality_tol)
477     date = args.date or _dt.date.today().isoformat()
478
479     if not os.path.exists(args.manifest):
480         log.error("manifest not found: %s", args.manifest)
481         return 2
482     manifest = load_manifest(args.manifest)
483     baseline = load_baseline(args.baseline)
484
485     try:
486         run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487     except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488         log.error("run failed: %s", e)
489         return 2
490     if not run_results:
491         log.error("no videos in manifest %s", args.manifest)
492         return 2
493
494     billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall

```

11. assistant (2026-06-22T09:45:50.775Z)

Let me check the manifest loading function to see if there's any schema validation:

12. user (2026-06-22T09:45:51.801Z)

```
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
```

13. assistant (2026-06-22T09:45:54.213Z)

Now let me check the backend.billing module to understand how it handles zero or negative rates:

14. user (2026-06-22T09:45:54.893Z)

backend\billing.py

15. user (2026-06-22T09:45:58.983Z)

```
1     """Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
2
3     ``cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)``
4     (the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
5     **USD**
6     (matches GCP billing = one source of truth) and **displayed in INR** at a configurable
7     FX rate
8     (the Bangalore market). The pricebook is a versioned DB table
9     (``backend/infrastructure/database.py``);
10    this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates
11    dict, so it
12    is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
13
14    **Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every
15    cost is
16    ``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated
17    pricebook version.
18    """
19
20    from __future__ import annotations
21
22    from typing import Dict, Mapping, Tuple
23
24    # Metered resource keys ? shared by the usage dict, the pricebook rows, and the
25    # breakdown.
26    GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second,
27    per gpu_type
28    WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't
29    billed as GPU)
30    EGRESS_GB = "egress_gb"          # reel/byte egress in GB
31    GEMINI_USD = "gemini_usd"        # provider-reported cost, ALREADY in USD (a passthrough,
32    not rate-×)
33
34    # Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a
35    USD amount).
36    _RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
37
38    def compute_cost_usd(usage: Mapping[str, float],
39                        rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
40        """Return ``(cost_usd, breakdown)`` for one job.
41
42        ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps
43        a resource
```

```

33         key to its USD unit price (already resolved for this pricebook version + gpu_type by
the DB
34         layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
35         Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total
is ``0.0``.
36         """
37         breakdown: Dict[str, float] = {}
38         total = 0.0
39         for res in _RATE_MULTIPLIED:
40             amount = float(usage.get(res, 0.0) or 0.0)
41             rate = float(rates.get(res, 0.0) or 0.0)
42             cost = round(amount * rate, 6)      # round the LINE, then sum rounded lines so
the
43             if amount or rate:                  # breakdown reconciles to the total exactly
(no drift),
44             breakdown[res] = cost              # and record the line even at $0 for
transparency
45             total += cost
46         gemini = round(float(usage.get(GEMINI_USD, 0.0) or 0.0), 6)
47         if gemini:
48             breakdown[GEMINI_USD] = gemini
49         total += gemini
50         return round(total, 6), breakdown
51
52
53     def apply_margin(cost_usd: float, margin: float) -> float:
54         """Resale price = ``cost_usd * (1 + margin)``. ``margin`` is clamped ? 0 (never
below cost)."""
55         return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
56
57
58     def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
59         """Display conversion: USD ? INR at the configured FX rate (2 dp, the display
currency)."""
60         return round(float(cost_usd) * float(fx_inr_per_usd), 2)
61

```

16. assistant (2026-06-22T09:46:08.990Z)

Now I have all the evidence I need. Let me verify the flow one more time to be thorough - trace what happens if `machine\_usd\_per\_hr` is 0 or negative:

The finding claims:

1. Line 79: `rate\_per\_sec = float(machine\_usd\_per\_hr) / 3600.0` ? if `machine\_usd\_per\_hr` is 0 or negative, the rate becomes 0 or negative
2. Line 497: `machine\_usd\_per\_hr=float(manifest.get("machine\_usd\_per\_hr", DEFAULT\_MACHINE\_USD\_PER\_HR))` ? there's no validation after this
3. The `load\_manifest()` function (line 378-380) does raw JSON parsing with no schema validation

Let me verify the actual impact by tracing through what happens:

****Scenario: machine_usd_per_hr = 0****

- Line 497: Gets value from manifest (or defaults to 0.35, which is positive)
- Line 495-500: Calls `cost\_report()`
- Line 79: `rate\_per\_sec = 0.0 / 3600.0 = 0.0`
- Line 82: Calls `billing.compute\_cost\_usd(usage, rates)` with rate `0.0`
- Line 42: `cost = amount \* rate = billed\_seconds \* 0.0 = 0.0`
- Result: `usd = 0.0` (misleading, suggests the run was free)

****Scenario: machine_usd_per_hr = -0.35 (malformed manifest)****

- Line 497: Gets value from manifest (or defaults to 0.35)
- Line 79: `rate\_per\_sec = -0.35 / 3600.0 = -0.0000972...`
- Line 82: `billing.compute\_cost\_usd()` with rate < 0
- Line 42: `cost = billed\_seconds \* (-rate) = NEGATIVE USD`

- Result: `usd < 0` (clearly wrong, would show negative cost in report)

Now let me verify there is NO validation anywhere:

17. user (2026-06-22T09:46:11.158Z)

No files found

18. assistant (2026-06-22T09:46:13.476Z)

Confirmed - there is NO validation of `machine\_usd\_per\_hr`. Now let me check if there's anything in the manifest schema or validation:

19. user (2026-06-22T09:46:14.572Z)

Found 1 file
scripts\nightly_reelcount.py

20. user (2026-06-22T09:46:17.753Z)

```
73: def cost_report(billed_seconds: float, machine_usd_per_hr: float,
79:     rate_per_sec = float(machine_usd_per_hr) / 3600.0
86:     "machine_usd_per_hr": float(machine_usd_per_hr),
234:     f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
497:     machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
```

21. assistant (2026-06-22T09:46:29.603Z)

Perfect. I have all the evidence. Let me verify the actual values in the manifest to see if this is a realistic concern:

The manifest (lines 9-10) shows:

- `"machine\_usd\_per\_hr": 0.70` (positive, good)
- `"fx\_inr\_per\_usd": 83.0` (positive, good)

However, the **real defect** is that:

1. The `load\_manifest()` function does **zero validation** on the manifest JSON
2. If a human (or a script) accidentally puts `"machine\_usd\_per\_hr": 0` or `"machine\_usd\_per\_hr": -0.35` in the manifest, there is **no guard** to catch it
3. The result would be:
 - `0` ? misleading cost report showing `\$0.00` (appears free when it shouldn't be)
 - Negative ? outright wrong cost report showing negative USD

The finding is **REAL**. This is a **low severity** issue because:

- The default value is positive (0.35)
- The example manifest has a positive value
- It requires a human error (typo in manifest JSON)
- But when it occurs, it silently produces incorrect cost reports

However, let me verify if there are any other layers of validation or if the billing module might reject negative rates:

Looking at `billing.compute\_cost\_usd()` (lines 28-50):

- Line 40-41: Converts `amount` and `rate` to floats, defaults missing to `0.0`
- Line 42: `cost = amount \* rate` ? **no validation** that rate ≥ 0
- Line 43-44: Records the line if `amount` or `rate` is truthy
- Result: Negative rates produce negative costs with no error

VERIFICATION: The finding is **REAL**.

The issue is:

- **File:** `scripts\nightly\_reelcount.py`

- **Line:** 497 (where the manifest value is used) and 79 (where the rate is computed)
- **Defect:** No validation that `machine\_usd\_per\_hr` is positive before using it to calculate cost
- **Impact:** Zero or negative machine_usd_per_hr produces incorrect cost reports (either \$0 or negative USD)
- **Severity:** Low (requires manifest corruption, but impacts report accuracy)
- **Fix:** Add validation immediately after line 497 (or in `cost\_report()` itself)

22. user (2026-06-22T09:46:36.972Z)

Structured output provided successfully

23. assistant (2026-06-22T09:47:08.418Z)

Summary

VERDICT: REAL DEFECT ? The finding is accurate.

Evidence

The cost report calculation in `scripts/nightly\_reelcount.py` lacks validation for the `machine\_usd\_per\_hr` parameter:

- Line 497:** The manifest value is loaded with zero validation:

```
python
machine_usd_per_hr=float(manifest.get("machine_usd_per_hr", DEFAULT_MACHINE_USD_PER_HR))
```
- Line 79:** Rate is computed without guards:

```
python
rate_per_sec = float(machine_usd_per_hr) / 3600.0
```
- No schema validation:** The `load\_manifest()` function (lines 378-380) is a raw JSON parser with no type or range checking.
- Silent propagation:** The `backend.billing.compute\_cost\_usd()` function multiplies the rate by usage seconds with no validation (line 42 in billing.py), so negative rates produce negative costs without errors.

Impact

- If `machine\_usd\_per\_hr = 0` (manifest typo): Cost report shows \$0.00, misleading the reader
- If `machine\_usd\_per\_hr < 0` (manifest corruption): Cost report shows negative USD, clearly wrong

Both scenarios corrupt the correctness of the nightly regression report's cost line, undermining the integrity of the summary metrics email sent to the owner.

Recommended Fix

Add a validation check immediately after line 497:

```
python
if machine_usd_per_hr <= 0:
    raise ValueError(f'machine_usd_per_hr must be > 0, got {machine_usd_per_hr}')
```